

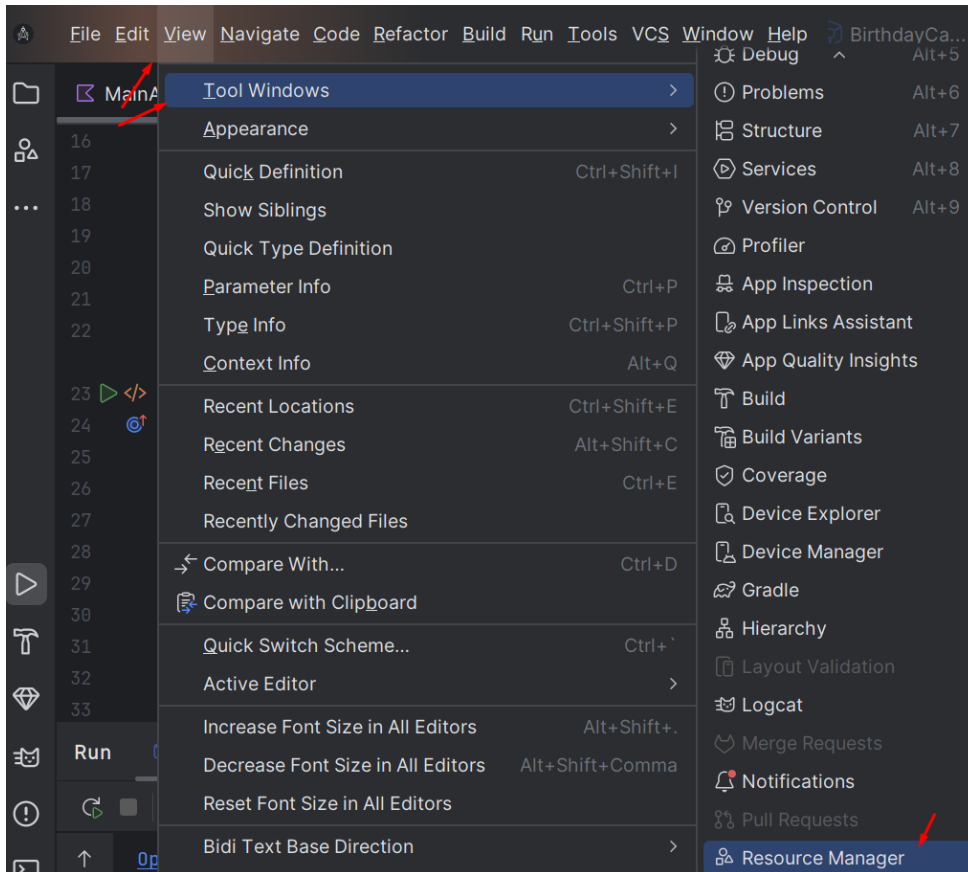
Лабораторная работа №3. Создание приглашения для героя

PixLetter: Часть II

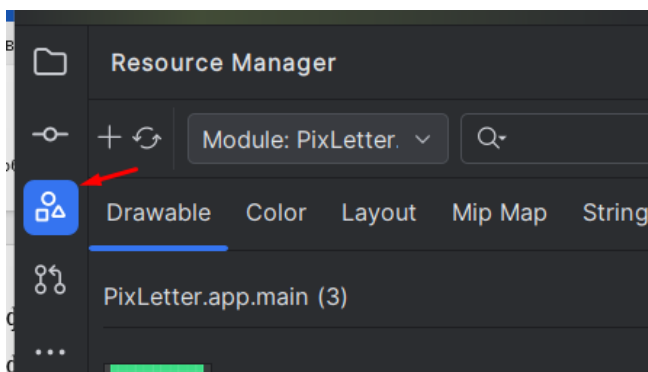
Шаг 13. Добавление изображения

1. Импорт изображения

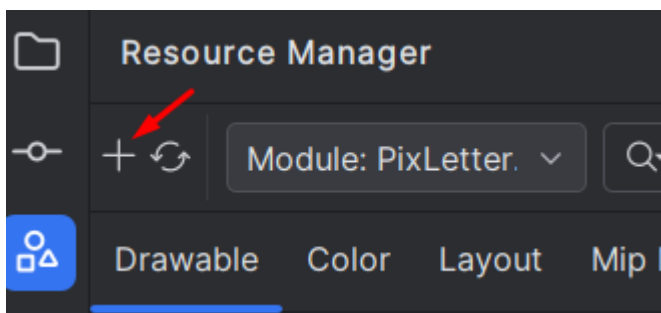
1. Откройте **Resource Manager: View > Tool Windows > Resource Manager**



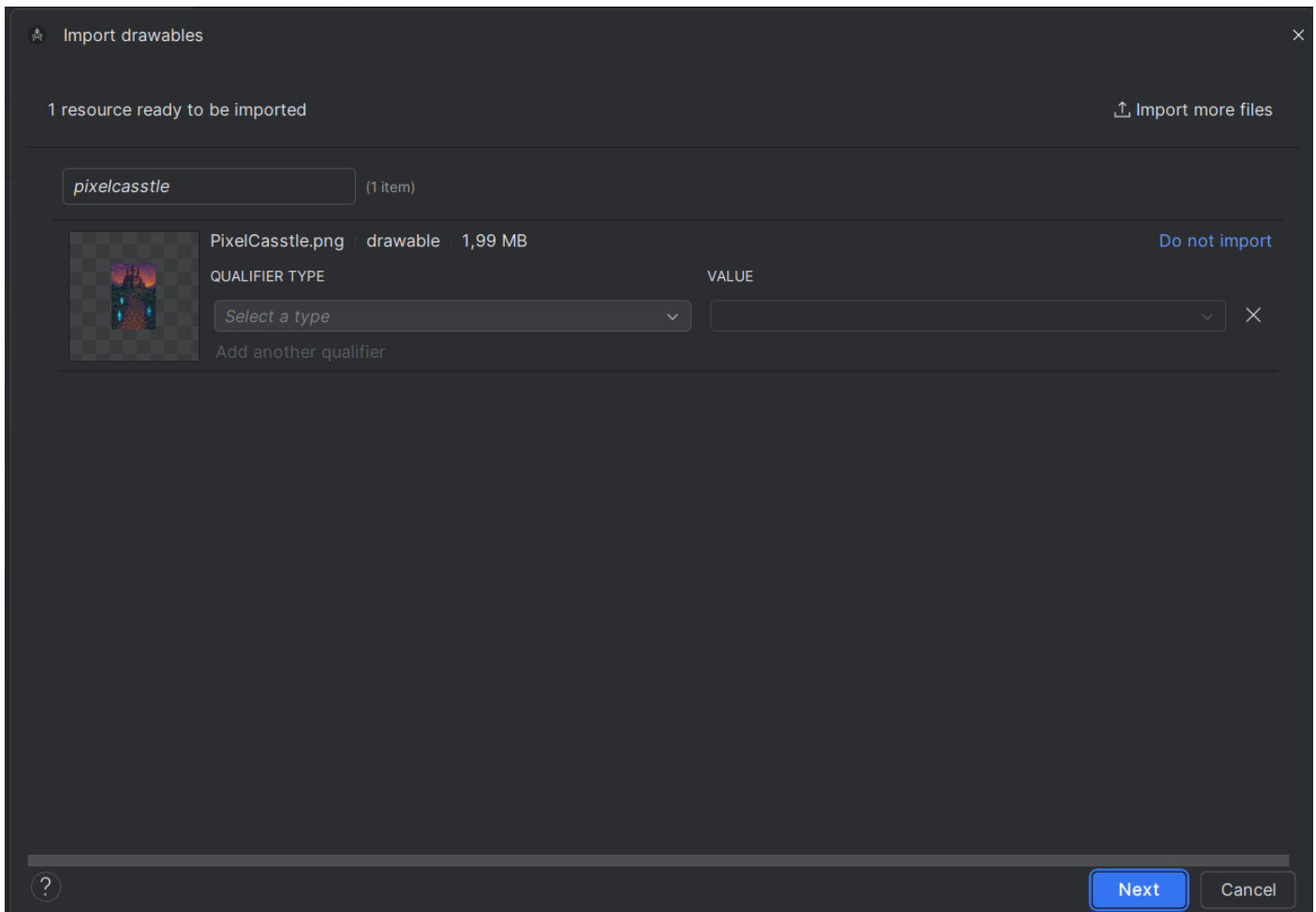
Или нажмите соответствующую вкладку:



2. Нажмите + → **Import Drawables**:

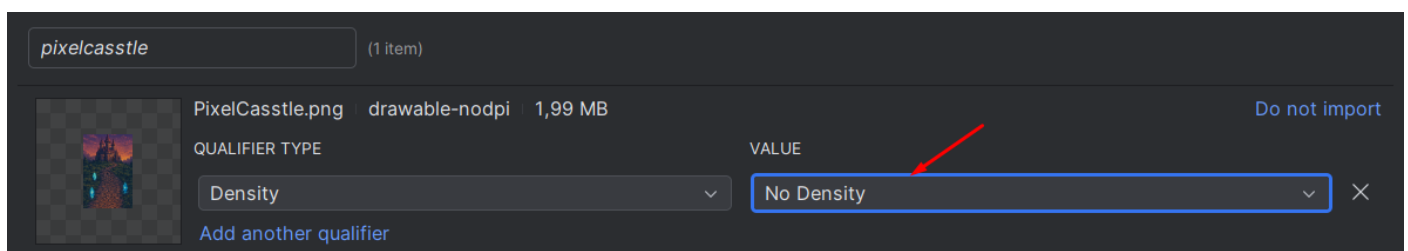
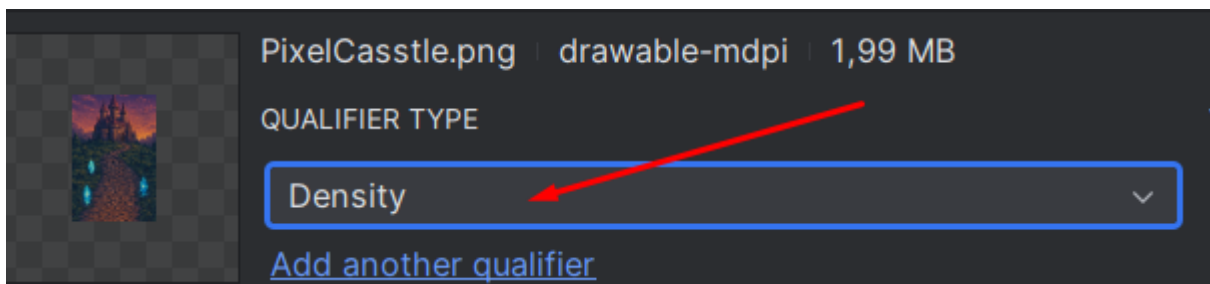


3. В файловом браузере выберите загруженный вами файл изображения и нажмите «**Open**». Это действие открывает диалоговое окно «**Import drawables**».



4. В диалоговом окне выберите:

- **Qualifier Type:** Density
- **Value:** No Density (чтобы избежать автоматического масштабирования).

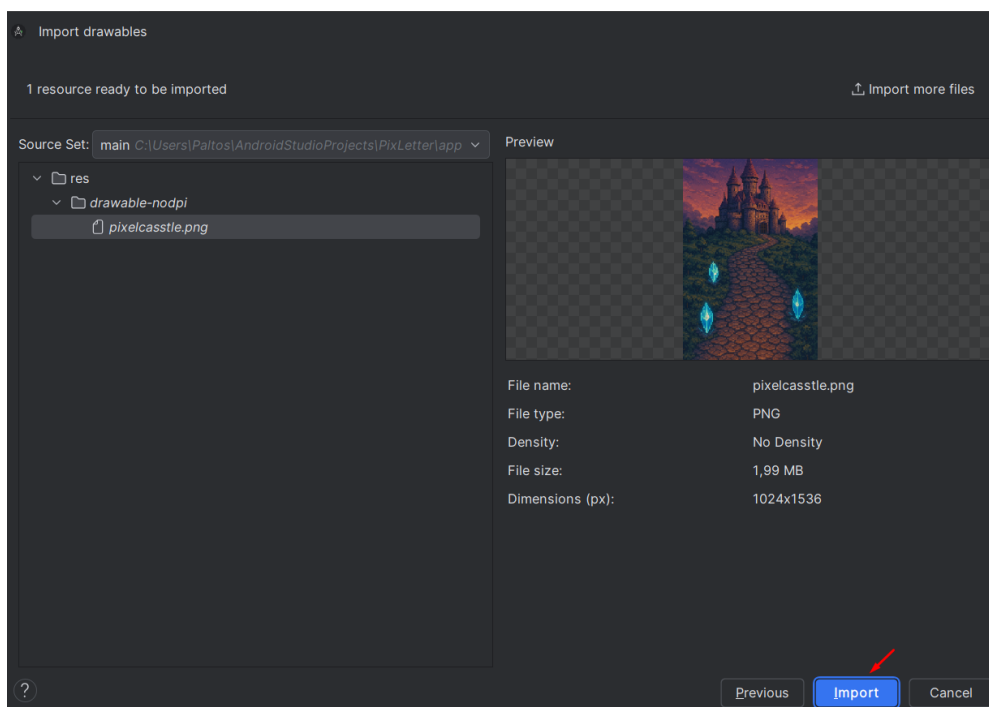


Устройства Android выпускаются с экранами разных размеров (телефоны, планшеты и телевизоры, и это лишь некоторые из них), и их экраны также имеют разные

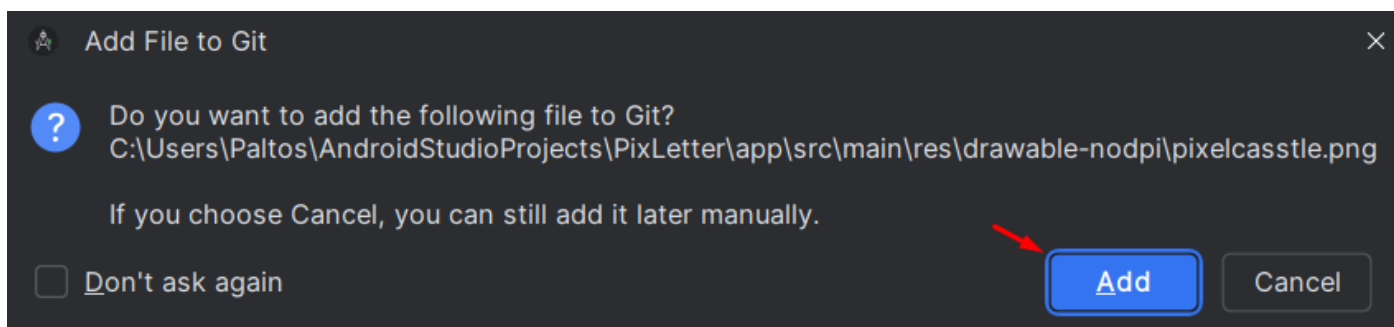
размеры пикселей. То есть, в то время как одно устройство имеет 160 пикселей на квадратный дюйм, другое устройство помещает 480 пикселей в то же пространство. Если вы не учитываете эти различия в плотности пикселей, система может масштабировать ваши изображения, что может привести к размытым изображениям или большим изображениям, которые потребляют слишком много памяти, или изображениям неправильного размера.

При изменении размера изображений, которые больше, чем может обработать система Android, выдается ошибка нехватки памяти. Для фотографий и фоновых изображений, таких как текущее изображение следует поместить их в `drawable-nodpi` папку, что остановит поведение изменения размера.

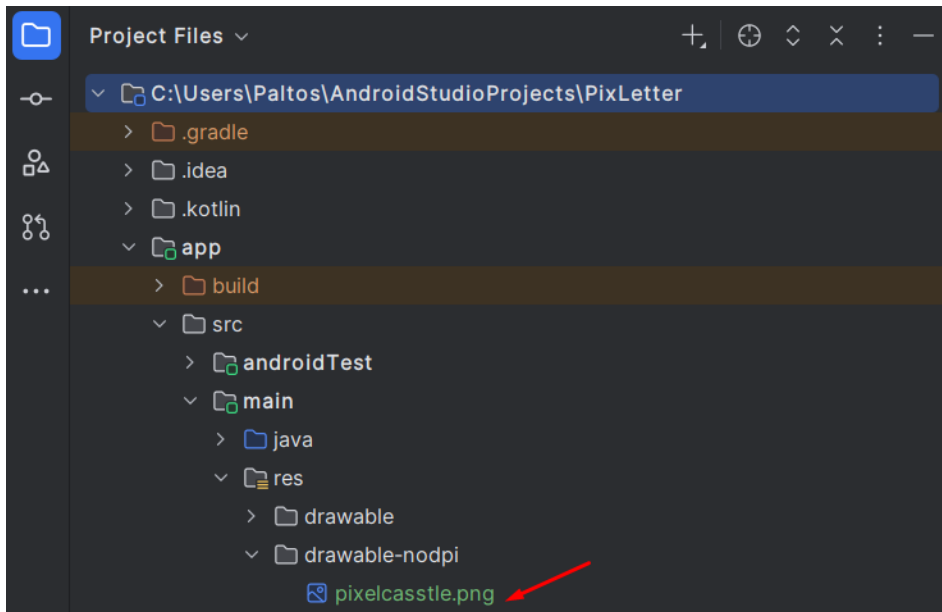
5. Нажмите **Next**. Android Studio показывает вам структуру папок, в которых будет размещено ваше изображение. Обратите внимание на **drawable-nodpi** папку. Нажмите **Import**:



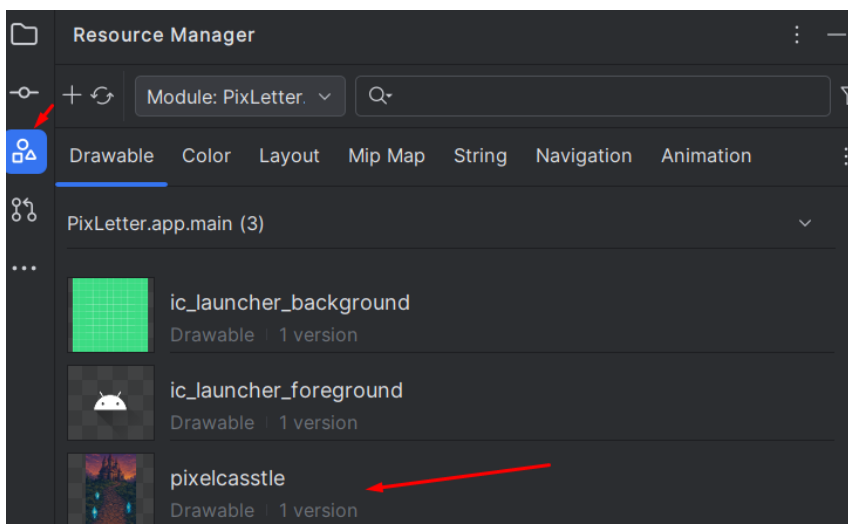
6. Подтвердите добавление файла в Git:



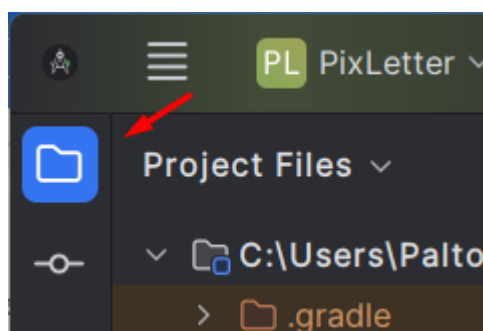
Android Studio создает drawable-nodpi папку и помещает в нее изображение. В представлении проекта Android Studio имя ресурса отображается как **pixelcasstle.png** (nodpi). В файловой системе компьютера Android Studio создала папку с именем drawable-nodpi:



Если изображение импортировано успешно, Android Studio добавляет его в список во вкладке **Drawable**. Этот список включает все ваши изображения и значки для приложения. Теперь вы можете использовать это изображение в своем приложении.



Вернитесь к представлению проекта, нажмите вкладку **Project** слева:



Шаг 14. Добавление composable изображения

Для отображения графики в приложении, аналогично текстовым элементам, **Compose** предоставляет компонент **Image**.

- Добавьте **@Composable** функцию **GreetingImage()** после функции **GreetingText()**.
- Передайте ей два **String** параметра: один **message** для текста письма, а другой **from** для подписи от короля:

```
@Composable
fun GreetingImage(message: String, from: String) {
}
```

- Каждая компонуемая функция должна принимать необязательный **Modifier** параметр. Модификаторы сообщают элементу пользовательского интерфейса, как размещать, отображать или вести себя в родительском макете. Добавьте еще один параметр в функцию **GreetingImage()**:

```
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
```

2. Работа с ресурсами в Compose

Ресурсы (изображения, строки и другие статические данные) должны храниться отдельно от кода для удобства поддержки и адаптации под разные устройства. В **Android** они размещаются в папке **res/**, которая содержит:

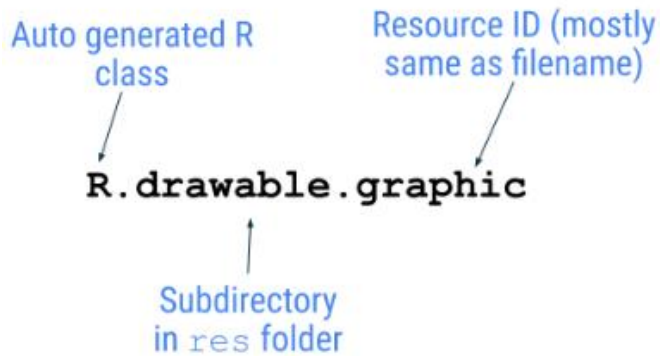
- **drawable/** — для изображений.
- **mipmap/** — для иконок приложения.
- **values/** — для строк и стилей.

Доступ к ресурсам

Доступ к ресурсам можно получить с помощью идентификаторов ресурсов, которые генерируются в **R** классе вашего проекта.

Класс **R** — это автоматически сгенерированный класс **Android**, содержащий идентификаторы всех ресурсов в проекте. В большинстве случаев идентификатор ресурса совпадает с именем файла. Например:

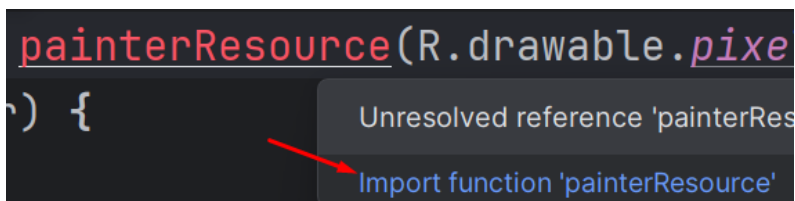
```
R.drawable.graphic
```



- В функции **GreetingImage()** объявите переменную **image**, используя **painterResource()** для загрузки изображения:

```
@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier) {
    val image = painterResource(R.drawable.pixelcasstle)
}
```

- **Android Studio** предложит импортировать необходимый пакет (**androidx.compose.ui.res.painterResource**).



Функция **painterResource()** загружает ресурс изображения и принимает идентификатор ресурса (**R.drawable.pixelcasstle** в данном случае) в качестве аргумента.

- Добавьте компонент **Image**, передав **image** в параметр **painter**.
- После вызова **painterResource()** функции добавьте **Image** составной объект, а затем передайте его **image** в качестве именованного аргумента для **painter**.
- Для декоративных изображений установите **contentDescription = null**, чтобы они игнорировались программами чтения экрана (например, **TalkBack**):

```
@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
    val image = painterResource(id = R.drawable.pixelcasstle)
    Image(
        painter = image,
        contentDescription = null
    )
}
```

4. Предварительный просмотр и запуск

В функции **RoyalInvitationPreview()** замените вызов **GreetingText()** на **GreetingImage()**.

Ваша функция должна выглядеть как этот фрагмент кода:

```
@Preview(showBackground = true)
@Composable
fun RoyalInvitationPreview() {
    PixLetterTheme {
        GreetingImage(
            message = "Royal Decree!\n Dear
                    great feast!\nCome to the castle
                    head!",
            from = "From King Arthur"
        )
    }
}
```

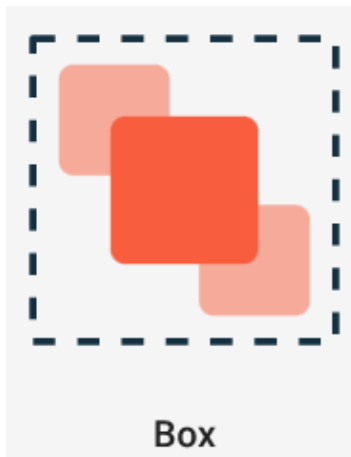
Проверьте результат в панели **Design**:



Шаг 15. Добавление Box layout

В **Jetpack Compose** три основных стандартных элемента макета — это **Column**, **Row** и **Box**. Ранее вы уже познакомились с **Column** и **Row**, теперь пришло время подробнее рассмотреть **Box**.

Box layout — один из стандартных элементов макета в **Compose**. Используйте **Box layout** для размещения элементов друг над другом. **Box layout** также позволяет вам настраивать определенное выравнивание элементов, которые он содержит.



В функции **GreetingImage()** оберните элемент **Image** в **Box**, как показано ниже:

```
@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
    val image = painterResource(id = R.drawable.pixelcasstle)
    Box {
        Image(
            painter = image,
            contentDescription = null
        )
    }
}
```

При необходимости Android Studio предложит вам импортировать **Box** из **androidx.compose.foundation.layout**

```
import androidx.compose.foundation.layout.Box
```

Модификаторы макета

Модификаторы позволяют настраивать внешний вид и поведение элементов пользовательского интерфейса. Например, с их помощью можно задать отступы (**padding**), выравнивание, фон и другие параметры.

Вы уже использовали модификатор **padding** ранее для добавления пространства вокруг **Text**. В дальнейшем вы будете применять модификаторы всё активнее.

Расположение и выравнивание элементов

Модификаторы можно применять не только к отдельным элементам, но и к макетам (Column, Row, Box) для позиционирования вложенных компонентов.

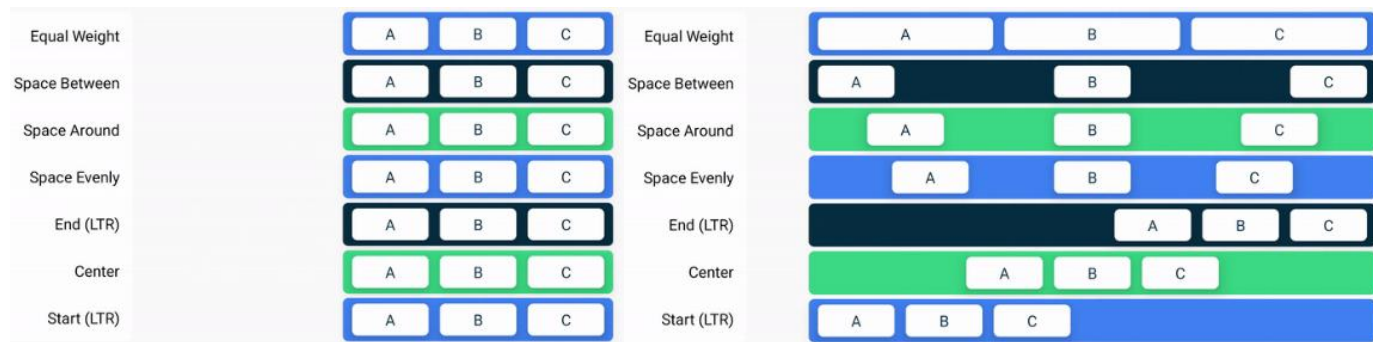
В Column:

Используйте **verticalArrangement**, чтобы задать вертикальное размещение дочерних элементов, когда доступное пространство больше, чем их совокупный размер.



В Row:

Используйте **horizontalArrangement**, чтобы аналогично задать горизонтальное размещение элементов.

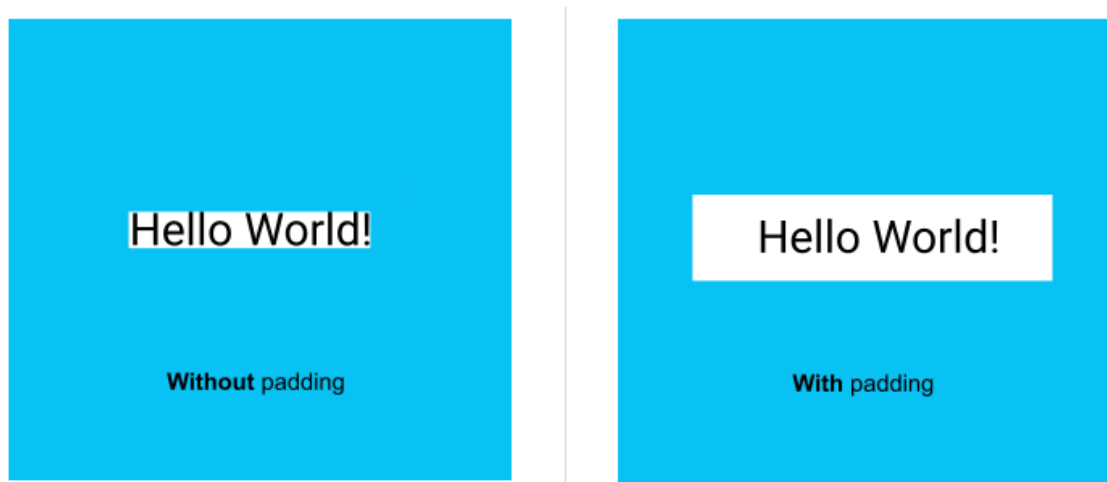


В обоих случаях:

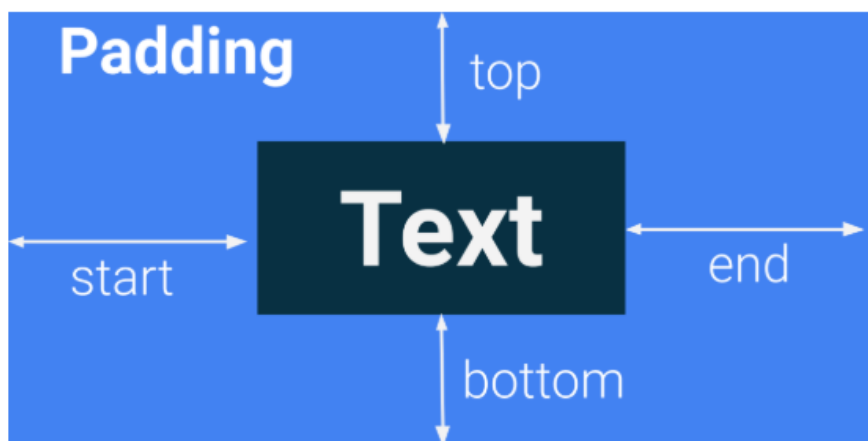
- Для выравнивания по краям, центру и т.д. применяйте параметр **horizontalAlignment** или **verticalAlignment**.

Padding

Каждый элемент интерфейса по умолчанию занимает только пространство, необходимое для его содержимого. Чтобы добавить отступы от границ, используйте модификатор `padding`.



`Padding` используется как модификатор, что означает, что вы можете применить его к любому компоненту. Для каждой стороны модификатор `padding` принимает необязательный аргумент, который определяет величину `padding`.



- Передайте параметр **modifier** в **Box**

```
Box(modifier) {
```

- А затем — добавим функцию **GreetingText()**, как показано ниже:

```
Box(modifier) {
    Image(
        painter = image,
        contentDescription = null
    )
    GreetingText(
        message = message,
        from = from,
        modifier = Modifier
            .fillMaxSize()
            .padding(all = 8.dp)
    )
}
```

Обратите внимание на обновленный предварительный просмотр на панели «**Design**». Вы должны увидеть текст и изображение:



Настройка цвета текста

Чтобы изменить цвет текста на **белый** как в **предварительном просмотре**, так и в **итоговом приложении**, необходимо добавить параметр **color = Color.White** в оба вызова **Text()** внутри функции **GreetingText()**.

Также не забудьте **импортировать Color** из **androidx.compose.ui.graphics.Color**.

```

@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Center,
        modifier = modifier
    ) {
        Text(
            text = message,
            fontSize = 20.sp,
            lineHeight = 40.sp,
            textAlign = TextAlign.Center,
            color = Color.White
        )
        Text(
            text = from,
            fontSize = 24.sp,
            color = Color.White,
            modifier = Modifier
                .padding(all = 16.dp)
                .align(alignment = Alignment.End)
        )
    }
}

```

Чтобы изменения отразились на экране, замените вызов **GreetingText()** в **onCreate()** на **GreetingImage()**:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PixLetterTheme {
                GreetingImage(
                    message = "Royal Decree!\n Dear Dendragon,\n Come to the castle before sunset,\n otherwise the kingdom will be in danger.",
                    from = "From King Arthur",
                    modifier = Modifier.padding(all = 8.dp)
                )
            }
        }
    }
}

```

Теперь изображение занимает всю ширину экрана, но прикреплено к его верхней части. Внизу остаётся пустое пространство, что выглядит не очень красиво. В

следующих шагах вы узнаете, как улучшить визуальное размещение и сделать интерфейс более сбалансированным.

Шаг 16. Изменение непрозрачности и масштаба изображения

Масштабирование содержимого изображения

Вы уже добавили изображение в приложение и разместили его. Теперь нужно настроить масштаб, чтобы изображение заполняло весь экран.

Свойство **contentScale** управляет тем, как изображение изменяет свой размер при размещении. Существует несколько доступных значений, и в этом случае вы используете:

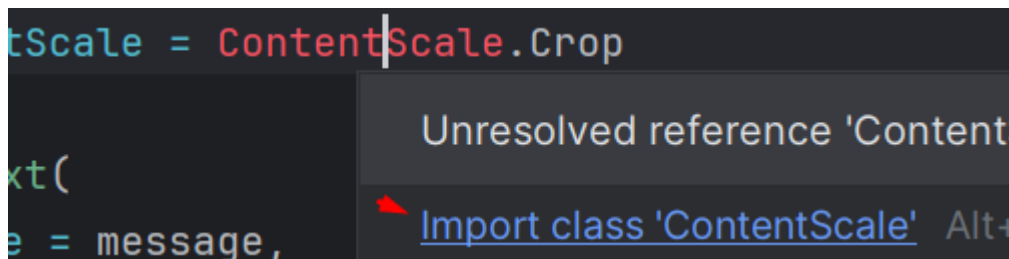
contentScale = ContentScale.Crop

Этот параметр масштабирует изображение, **сохраняя соотношение сторон**, так, чтобы **ширина и высота** изображения были **равны или больше** размеров контейнера. Это означает, что изображение может быть **обрезано по краям**, чтобы вписаться в отведённую область.

Добавьте параметр **contentScale = ContentScale.Crop** в вызов **Image()**:

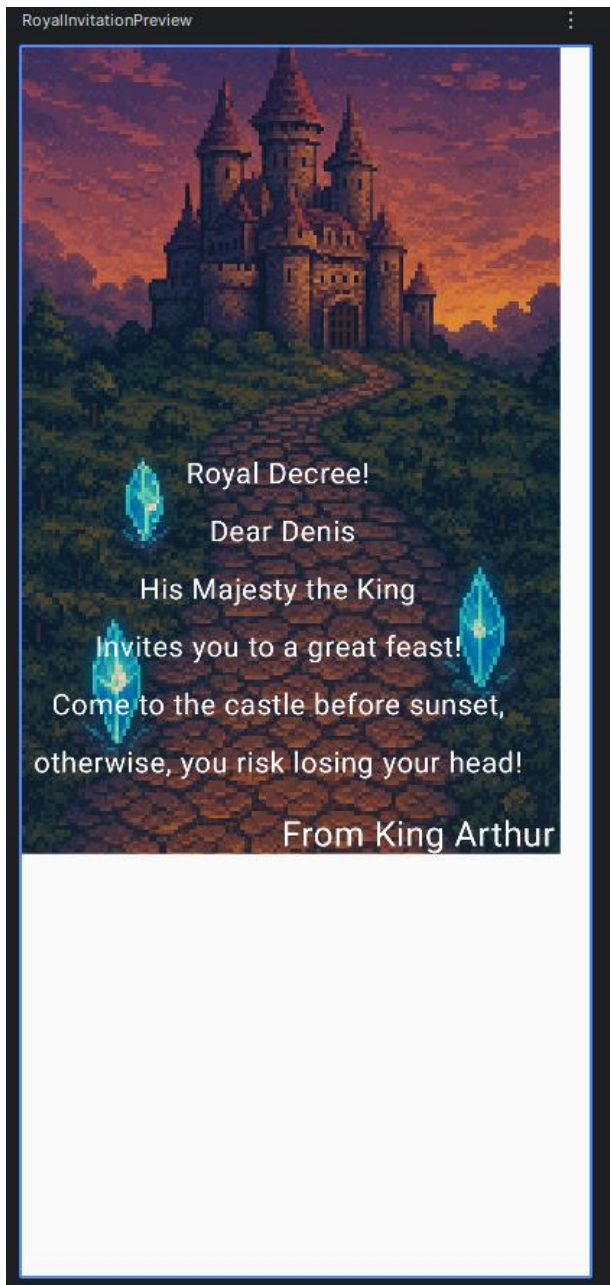
```
@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
    val image = painterResource(id = R.drawable.pixelcasstle)
    Box(modifier) {
        Image(
            painter = image,
            contentDescription = null,
            contentScale = ContentScale.Crop,
        )
    }
}
```

Android Studio может подсветить **ContentScale**. В этом случае согласитесь на импорт:



The screenshot shows a code editor with the line `contentScale = ContentScale.Crop`. A red squiggly line under `ContentScale` indicates an unresolved reference. A tooltip is displayed, showing the text "Unresolved reference 'ContentScale'" and a blue link "Import class 'ContentScale' Alt+I". A red arrow points to the link.

Обратите внимание что изображение не заполнило полностью экран:



В `Image()` параметр `contentScale = ContentScale.Crop` не гарантирует, что изображение будет полностью растянуто по всей области. Вот почему:

Что делает `ContentScale.Crop`

- Он масштабирует изображение, чтобы оно заполнило всё доступное пространство, сохраняя соотношение сторон.
- Если размеры `Image` больше, чем изображение по одной из осей, часть изображения может быть обрезана.

Сейчас мы можем добавить модификатор `fillMaxSize()`:


```

@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
    val image = painterResource(id = R.drawable.pixelcasstle)
    Box(modifier) {
        Image(
            painter = image,
            contentDescription = null,
            contentScale = ContentScale.Crop,
            modifier = Modifier.fillMaxSize(),
        )
    }
}

```

Обратите внимание как теперь выглядит изображение. В нашем случае **ContentScale.Crop** режет изображение неудачно, вот несколько альтернатив, которые позволят по-другому отобразить фон, не обрезая важные части:

Теперь изображение должно заполнить весь экран предварительного просмотра, как вы можете видеть на этом снимке экрана:

ContentScale.Fit

- Не обрезает изображение.
- Сохраняет соотношение сторон.
- Может оставить **пустые поля** (черные/прозрачные) по краям, если пропорции не совпадают.



ContentScale.FillBounds

- Растягивает изображение на **весь контейнер**.
- Может **искажить** изображение (соотношение сторон не сохраняется), но ничего не обрезается.



ContentScale.FillHeight или FillWidth

- Масштабирует изображение до **высоты** или **ширины** контейнера.
- Остальное может быть обрезано, но **меньше, чем у Crop**.



Выберите вариант, который понравился вам больше всего.

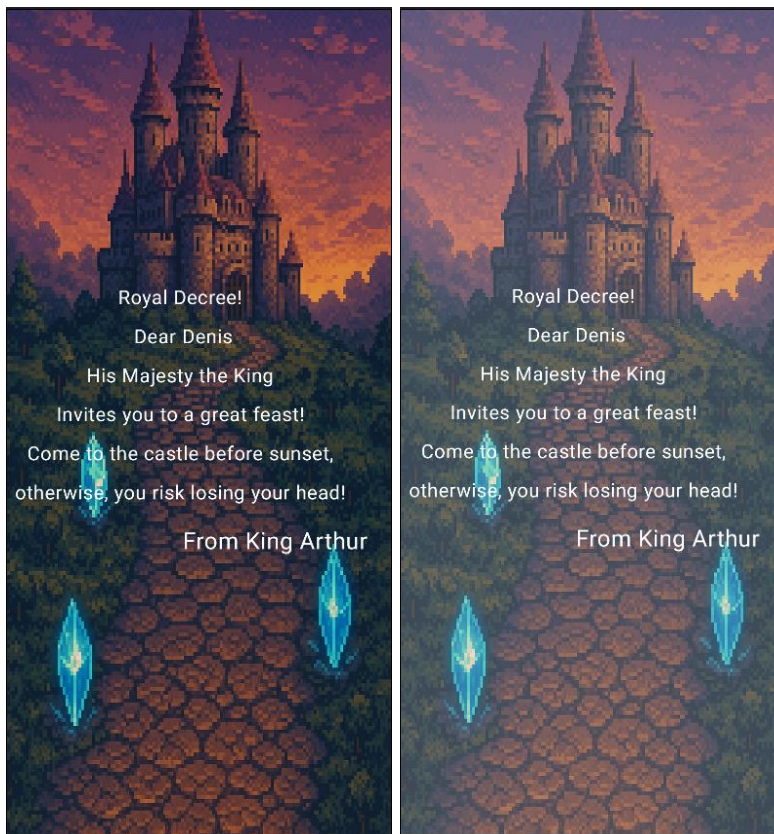
Изменение непрозрачности фонового изображения

Чтобы улучшить читаемость текста и контрастность, вы можете **изменить прозрачность фонового изображения**.

Добавьте параметр **alpha** в **Image** и установите значение 0.8F:

```
Image(  
    painter = image,  
    contentDescription = null,  
    contentScale = ContentScale.FillBounds,  
    alpha = 0.8F,  
    modifier = Modifier.fillMaxSize(),  
)
```

Это сделает изображение слегка прозрачным, чтобы текст поверх него выглядел лучше:



Запуск приложения

Теперь запустите приложение на эмуляторе или физическом устройстве и убедитесь, что изображение и текст отображаются корректно.

Шаг 17. Практика хорошего кода: Извлечение строк

При разработке приложений важно предусмотреть возможность **перевода** интерфейса. Чтобы упростить локализацию, избегайте жёстко закодированных строк — строк, прописанных напрямую в коде.

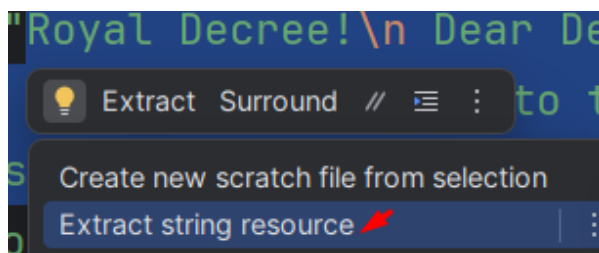
Вместо этого **извлекайте текст в строковые ресурсы (strings.xml)**. Это:

- облегчает перевод,
- упрощает повторное использование текста в разных местах,
- делает код чище.

Внутри **onCreate()** выделите строку с текстом письма, но **без кавычек**:

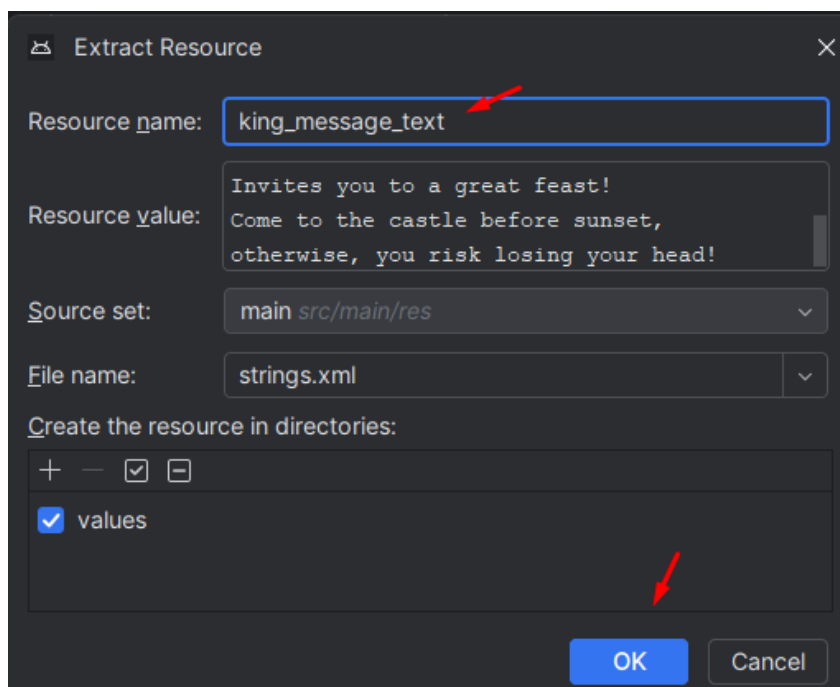
```
GreetingImage(  
    message = "Royal Decree!\n Dear Denis,\nHis  
Majesty the King\nInvites you to a great  
feast!\nCome to the castle before sunset,  
\notherwise, you risk losing your head!",  
    from = "From King Arthur",
```

Нажмите на лампочку  слева от строки. Выберите **Extract string resource**:



В открывшемся диалоговом окне:

- В поле **Resource name** введите **king_message_text**
- Убедитесь, что **Resource value** содержит нужный текст
- Нажмите **OK**



Android Studio создаст строковый ресурс и автоматически обновит ваш код, заменив строку на:

```
PixLetterTheme {
    GreetingImage(
        message = stringResource(id = R.string.king_message_text),
```

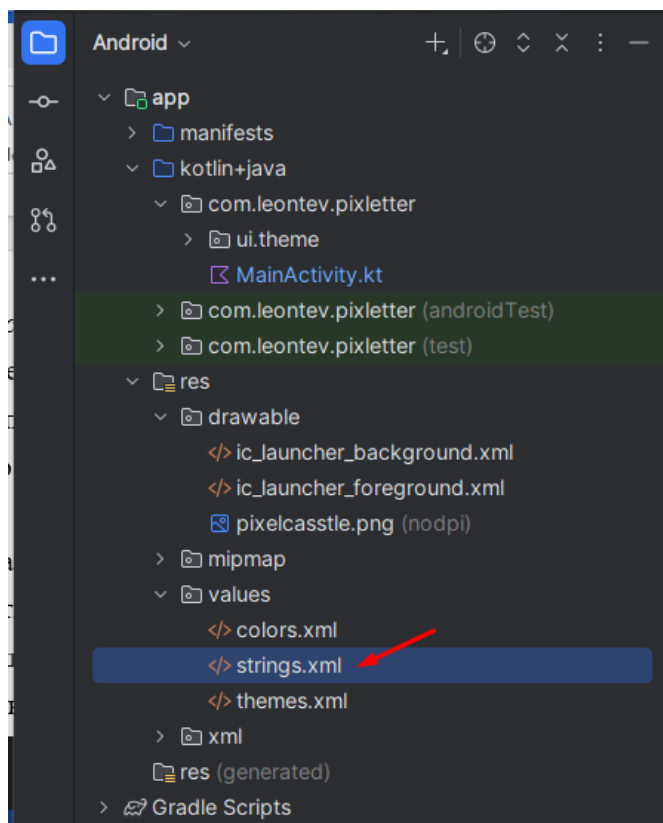
Обратите внимание на изменения в коде.

При необходимости добавьте импорт:

```
import androidx.compose.ui.res.stringResource
```

Проверка файла strings.xml

1. Перейдите в панель **Project**
2. Откройте файл strings.xml по пути: **app > res > values > strings.xml**



Вы увидите, что строка **king_message_text** добавлена:

```
<resources>
    <string name="app_name">PixLetter</string>
    ⚡ <string name="king_message_text">Royal Decree!\n Dear
    ↵ Denis\nHis Majesty the King\nInvites you to a great
    ↵ feast!\nCome to the castle before sunset,\notherwise, you
    ↵ risk losing your head!</string>
</resources>
```

Также обратите внимание: имя вашего приложения и другие строки уже хранятся здесь — всё в одном месте, удобно для локализации и поддержки.

Повторите те же действия, чтобы извлечь строку с подписью:

1. Выделите текст подписи в коде
2. Нажмите на лампочку
3. Выберите **Extract string resource**
4. Убедитесь, что поле **Resource name** — это **from_king_arthur**
5. Нажмите **OK**

```
PixLetterTheme {  
    GreetingImage(  
        message = stringResource(id = R.string.king_message_text),  
        from = "From King Arthur",  
        modifier = Modifier.padding(all = 8.dp)  
    )  
}
```

Ваш готовый файл должен выглядеть как этот фрагмент кода:

```
<resources>  
    <string name="app_name">PixLetter</string>  
    <string name="king_message_text">Royal Decree!\n Dear Denis\nHis Majesty the King\nInvites  
you to a great feast!\nCome to the castle before sunset,\notherwise, you risk losing  
your head!</string>  
    <string name="from_king_arthur">From King Arthur</string>  
</resources>
```

В функции **RoyalInvitationPreview()** обновите вызов **GreetingImage()**, чтобы использовать строки из ресурсов:

```
@Preview(showBackground = true)  
@Composable  
fun RoyalInvitationPreview() {  
    PixLetterTheme {  
        GreetingImage(  
            message = stringResource(id = R.string.king_message_text),  
            from = stringResource(id = R.string.from_king_arthur)  
        )  
    }  
}
```

Перезапустите приложение. Убедитесь, что всё работает так же, как раньше — только теперь строки вынесены в ресурсы, а изображение стало чуть прозрачнее для лучшей читаемости текста.

Шаг 18. Смещение текста в макете

Если вы хотите **поднять или опустить текст** относительно изображения на экране, вы можете использовать параметры **verticalArrangement** в **Column**, а также **Modifier.align()** и **Modifier.padding()** в комбинации с **Box**.

По умолчанию **Column** размещает текст по центру вертикально. Вы можете изменить его с помощью **verticalArrangement**.

1. Размещение текста ближе к верху:

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Top,
        modifier = modifier
    )
}
```

2. Размещение текста ближе к низу:

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Bottom,
        modifier = modifier
    )
}
```

3. Точная настройка отступов:

Можно добавить **Modifier.padding(top = ..., bottom = ...)**, чтобы задать конкретное смещение.

```
Column(
    verticalArrangement = Arrangement.Top,
    modifier = modifier
        .fillMaxSize()
        .padding(top = 10.dp)
) {
}
```

Шаг 19. Добавление рамки (границы) вокруг текста

Добавим рамку к **Column**, которая содержит **Text**-элементы, в функции **GreetingText**.

```

@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Top,
        modifier = modifier
        .border(
            width = 2.dp,
            color = Color.White,
            shape = RoundedCornerShape(12.dp)
        )
        .padding(all = 16.dp)
    )
}

```

Добавьте недостающие импорты.

Шаг 20. Создание отдельной функции для заголовка приглашения

Создайте отдельную функцию InvitationTitle()

Добавьте под остальными функциями новую **@Composable** функцию, которая отображает заголовок письма:

```

@Composable
fun InvitationTitle(modifier: Modifier = Modifier) {
    Text(
        text = stringResource(id = R.string.invitation_title),
        fontSize = 28.sp,
        color = Color.Yellow,
        textAlign = TextAlign.Center,
        modifier = modifier
        .padding(bottom = 16.dp)
    )
}

```

Здесь:

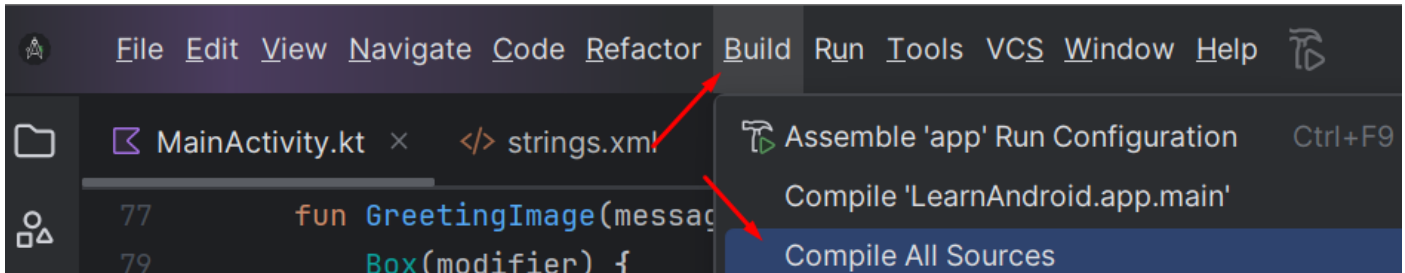
- **fontSize = 28.sp** — делает заголовок крупным.
- **Color.Yellow** — хорошо выделяется на фоне.
- **textAlign = TextAlign.Center** — выравнивание по центру.
- **padding(bottom = 16.dp)** — добавляет отступ снизу.

Добавьте строковый ресурс

Откройте файл **res/values/strings.xml** и добавьте новый элемент **invitation_title**, при этом не забудьте удалить из **king_message_text** тот же самый текст:

```
<resources>
  <string name="app_name">PixLetter</string>
  <string name="king_message_text">Royal Decree!\n Dear Denis
  your head!</string>
  <string name="from_king_arthur">From King Arthur</string>
  <string name="invitation_title">Royal Decree!</string>
</resources>
```

Если у вас компилятор ругается на ошибки, перекомпилируйте проект:



Вызовите функцию InvitationTitle в GreetingText

Теперь обновите функцию **GreetingText**, чтобы в начале столбца появился заголовок:

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Top,
        modifier = modifier
        .border(
            width = 2.dp,
            color = Color.White,
            shape = RoundedCornerShape(size = 12.dp)
        )
        .padding(all = 16.dp)
    ) {
        InvitationTitle()
```

Для выравнивания элемента по центру давайте используем **fillMaxWidth()**. Он делает текст таким широким, как позволяет контейнер (Column):


```
@Composable
fun InvitationTitle(modifier: Modifier = Modifier) {
    Text(
        text = stringResource(id = R.string.invitation_title),
        fontSize = 28.sp,
        color = Color.Yellow,
        textAlign = TextAlign.Center,
        modifier = modifier
            .fillMaxWidth()
            .padding(bottom = 16.dp)
    )
}
```

